How AIStudio Generates Images

A local, two-stage pipeline: a language model plans the picture, a diffusion model paints it

The idea in one paragraph

AIStudio splits image generation into two jobs that suit two very different kinds of AI. A **large language model** (running in Ollama) is good at understanding a casual request and turning it into precise instructions. A **diffusion model** (FLUX, running in ComfyUI) is good at turning precise instructions into pixels, but poor at interpreting vague ones. The controller script joins the two, and everything runs on the Mac with no cloud service involved.

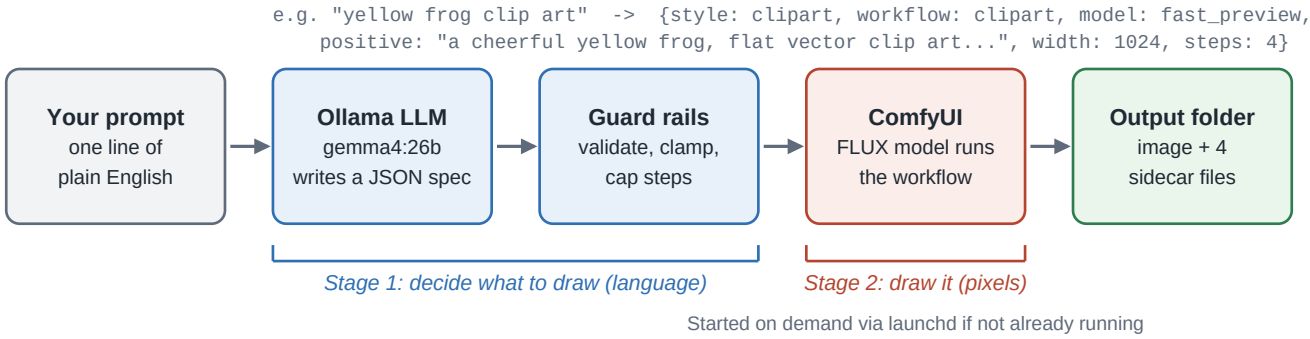


Figure 1. The AIStudio pipeline, from one-line prompt to saved image.

Stage 1: the LLM as art director

You type something short, such as *yellow frog clip art*. The controller sends it to gemma4:26b with a system prompt asking for a strict JSON specification: the style (photo, clip art, illustration or logo), which workflow and model to use, the image dimensions, step count and guidance strength, and an expanded positive prompt written the way diffusion models respond to best. Temperature is set to zero, so the same request always produces the same plan.

The LLM never chooses file names directly. It picks *capabilities* such as *fast_preview* or *photorealistic*, and config.yaml maps those to real model files. Swapping in a better model later is a one-line change.

Guard rails

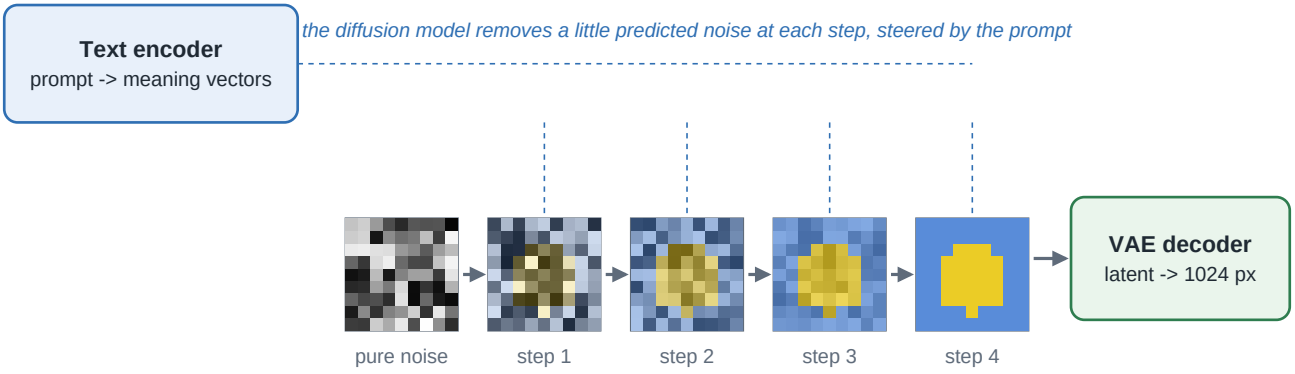
Language models occasionally return something odd, so the spec is checked before anything expensive happens. Missing keys stop the run; out-of-range numbers are clamped; a clip-art request is forced onto the clip-art workflow even if the LLM suggested a photographic one; and distilled models are capped at the step count they were trained for.

Stage 2: ComfyUI and the workflow graph

ComfyUI represents a generation as a graph of nodes: load the model, encode the prompt, create a blank canvas, sample, decode, save. AIStudio keeps one graph per job type as a JSON template, fills in the prompt, size, seed and steps from the spec, and submits it to ComfyUI's local API. If ComfyUI isn't running, the controller starts it through its LaunchAgent while the LLM is still thinking.

How diffusion actually makes the picture

The diffusion model starts from random noise and repeatedly predicts which part of it is noise, subtracting a little at each step. A text encoder converts the prompt into numbers that steer every prediction toward an image that matches it. The work happens in a compressed latent space; only at the end does a VAE decoder expand the result into a full-resolution image.

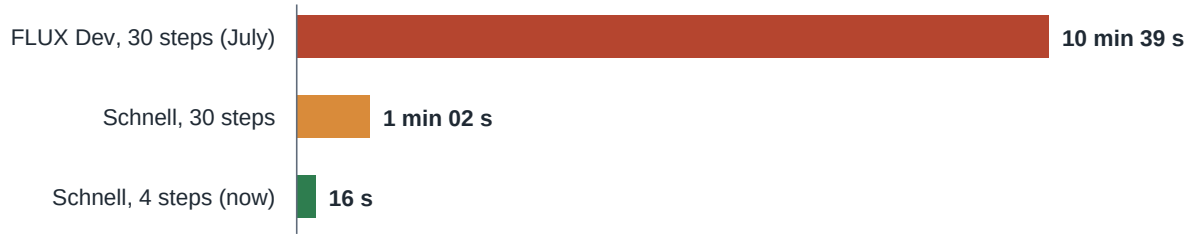


All of this happens in a compact 'latent' space, 1/8 of the final resolution, which is why it fits on a Mac.

Figure 2. Denoising from random noise to an image, guided by the encoded prompt.

Dev versus Schnell: why steps matter

Each step is a full pass through a 12-billion-parameter model, so step count dominates run time. FLUX Dev is the higher-quality model and needs 20 to 30 steps. FLUX Schnell is a distilled version, trained to reach a clean image in about four; giving it more simply wastes time. Capping Schnell at four steps took a clip-art image from a minute to sixteen seconds.



Measured ComfyUI time for one 1024 px image on the Mac Mini M4 Pro (excludes ~8 s of LLM time)

Figure 3. The effect of model choice and step count.

What you get back

Each run creates a dated folder, such as *2026-10-27 Green Frog Clip Art*, holding the image plus four sidecars: the LLM's spec (Prompt.json), the exact graph sent to ComfyUI (Workflow.json), the models, seed and timings used (Metadata.json), and a timestamped Generation.log. Together they let any image be reproduced exactly, in the same spirit as the photography pipeline's own logs.